



Database Management Systems

Functions

Dr. Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Functions



- We have already seen several functions that are built into the SQL language.
- In this section, we show how developers can write their own functions and procedures, store them in the database, and then invoke them from SQL statements.
- Functions and procedures allow “business logic” to be stored in the database and executed from SQL statements.
- These can be defined either by the procedural component of SQL or by an external programming language such as Java, C, or C++.
- Functions that are created by the user in the system database or a user-defined database are known as user-defined functions.
- The UDF functions accept parameters, perform actions, and returns the result.

Database Management Systems

Functions

- These functions help us to simplify our development by encapsulating complex business logic and making it available for reuse anywhere based on the needs.
- The user-defined functions make the code needed to query data a lot easier to write.
- They also improve query readability and functionality, as well as allow other users to replicate the same procedure.



- function in SQL Server always accepts parameters, either single or multiple and returns a single value or table.
- The functions are useful in the simplification of our code.
- Suppose we might have a complex computation that appears in a number of queries.
- In such a case, we can build a function that encapsulates the formula and uses it in each query instead of in each query.

Syntax:

```
CREATE FUNCTION schema_name.function_name (parameter_list)
RETURNS data_type AS
BEGIN
    statements
    RETURN value
END
```

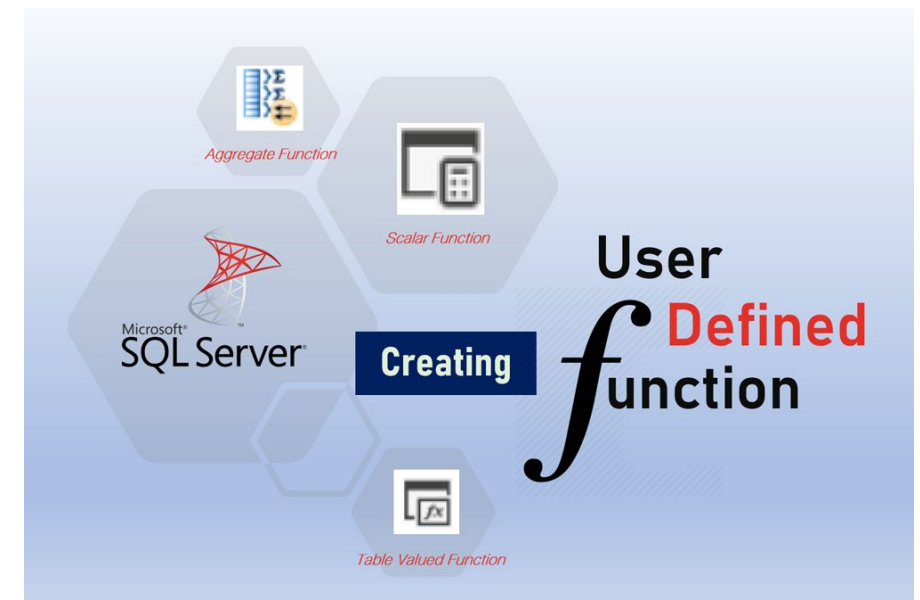
Database Management Systems

Functions

- **function_name** : It is the name of the function that needs to be created in MySQL.
- **parameter1, parameter2,...** : We can pass the optional parameters to the functions that need to be declared while creating it in the () brackets.
- **A function can contain none, one or more than one parameter.**
- **BEGIN and END** – BEGIN keyword marks the beginning of the function while END marks the completion of function in MYSQL.

How to Delete function?

```
DROP FUNCTION IF EXISTS function_name;
```



- **RETURN Datatype** – We can return any type of value from the execution of the function.
- The type of value that will be returned needs to be specified after the RETURN clause.
- Once, MySQL finds the RETURN statement while execution of the function, execution of the function is terminated and the value is returned.
- **DETERMINISTIC** – The function can be either deterministic or non-deterministic which needs to be specified here.
- When the function returns the same value for the same values of parameter then it is called deterministic. However, if the function returns a different value for the same values of functions then we can call that function to be nondeterministic.
- Functions will be NON-DETERMINISTIC by default.

Example:

```
CREATE FUNCTION Dept_size(deptno INT)
  RETURNS VARCHAR(7)
  BEGIN
    DECLARE No_of_emps INT;
    SELECT COUNT(*) INTO No_of_emps FROM EMPLOYEE WHERE Dno = deptno;
    IF No_of_emps > 3 THEN RETURN 'HUGE';
    ELSEIF No_of_emps > 2 THEN RETURN 'LARGE';
    ELSEIF No_of_emps > 1 THEN RETURN 'MEDIUM';
    ELSE RETURN 'SMALL';
    END IF;
  END
```

Query: select Dname,Dnumber,Dept_size(Dnumber)from department;

Output:

Dname	Dnumber	Dept_size(Dnumber)
Administration	4	HUGE
Headquarters	1	SMALL
Research	5	HUGE

Database Management Systems

Functions



Example: Write a function to find the age of employees

```
CREATE FUNCTION no_of_years(date1 date)
RETURNS INTEGER deterministic
BEGIN
    DECLARE date2 DATE;
    Select current_date()into date2;
    RETURN year(date2)-year(date1);
END
```

Query: select Fname,no_of_years(Bdate) from employee;

Output:

Fname	no_of_years(Bdate)
John	58
Franklin	68
Joyce	51
Richard	61
Ramesh	61
James	86
Jennifer	82
Ahmed	54
Alicia	55



THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu